



湖南大学
HUNAN UNIVERSITY

贪心算法

湖南大学计算机学院

目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度问题

第六节

找零问题

第七节

贪心算法与人工智能算法

概述-初识贪心算法



1元、2元、5元、10元、20元、50元、100元的纸币分别有若干张。

现在要用这些钱来支付 K 元，至少要用多少张纸币？

概述



贪心法：将问题的求解过程看作是一系列选择,每次选择一个输入,每次选择都是当前状态下的最好选择(局部最优解).每作一次选择后,所求问题会简化为一个规模更小的子问题.从而通过每一步的最优解逐步达到整体的最优解。

[常见应用] 背包问题,最小生成树,最短路径,作业调度等等。

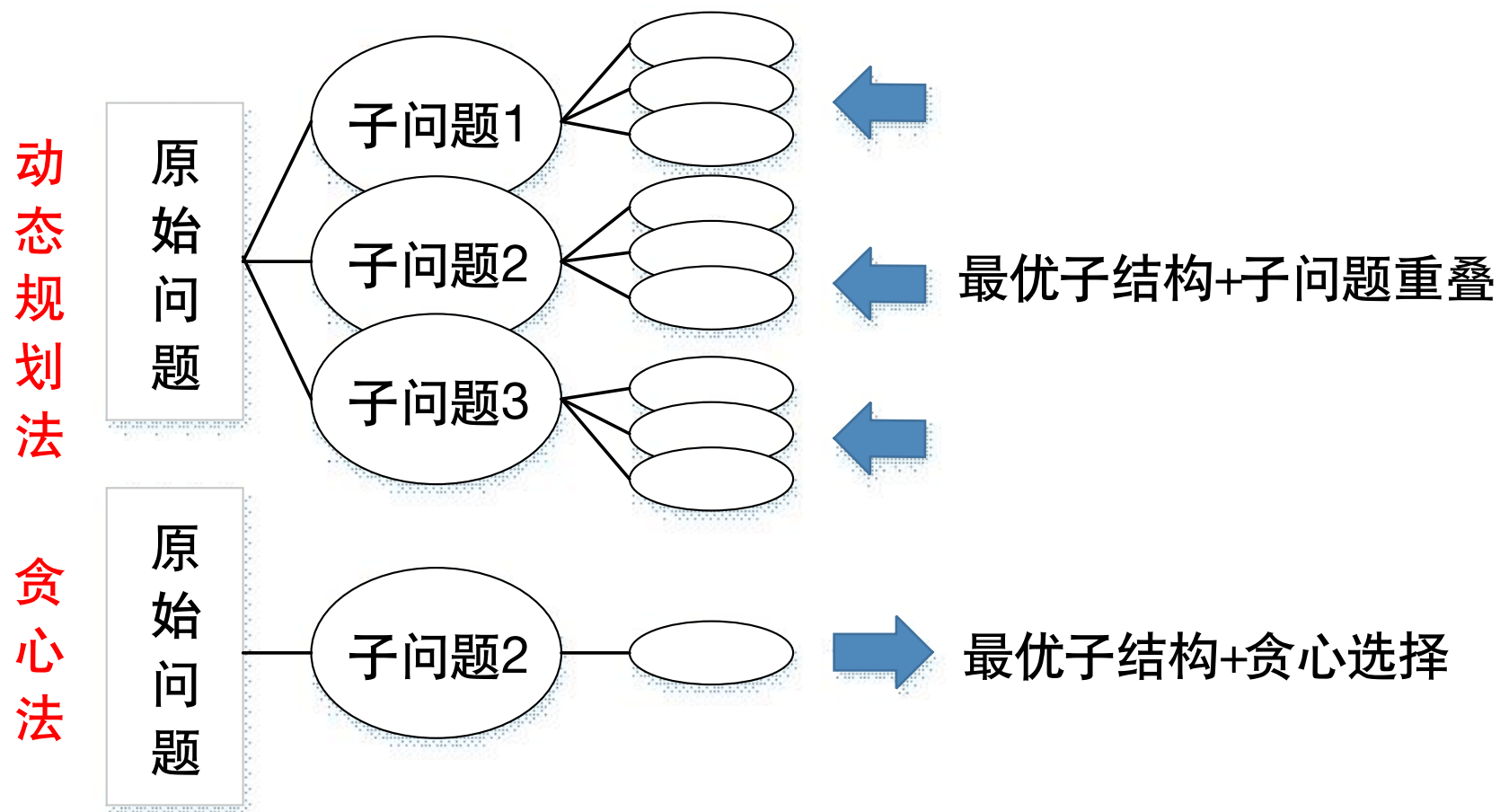
[算法优点] 求解速度快,时间复杂性有较低的阶。

[算法缺点] 需证明是最优解。

贪心算法原理



贪心算法 vs 动态规划法





目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度问题

第六节

找零问题

第七节

贪心算法与人工智能算法

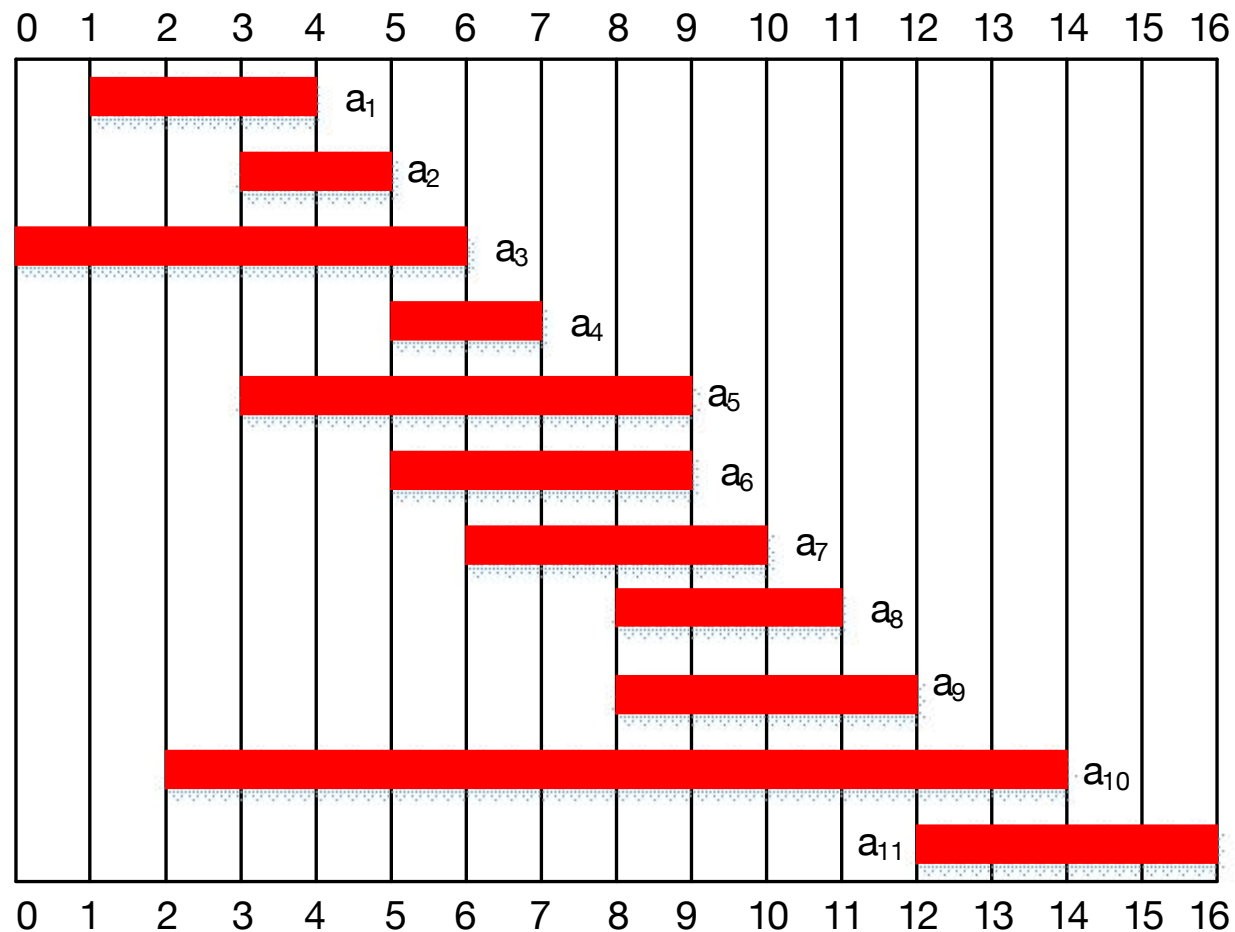
活动选择问题



问题描述:

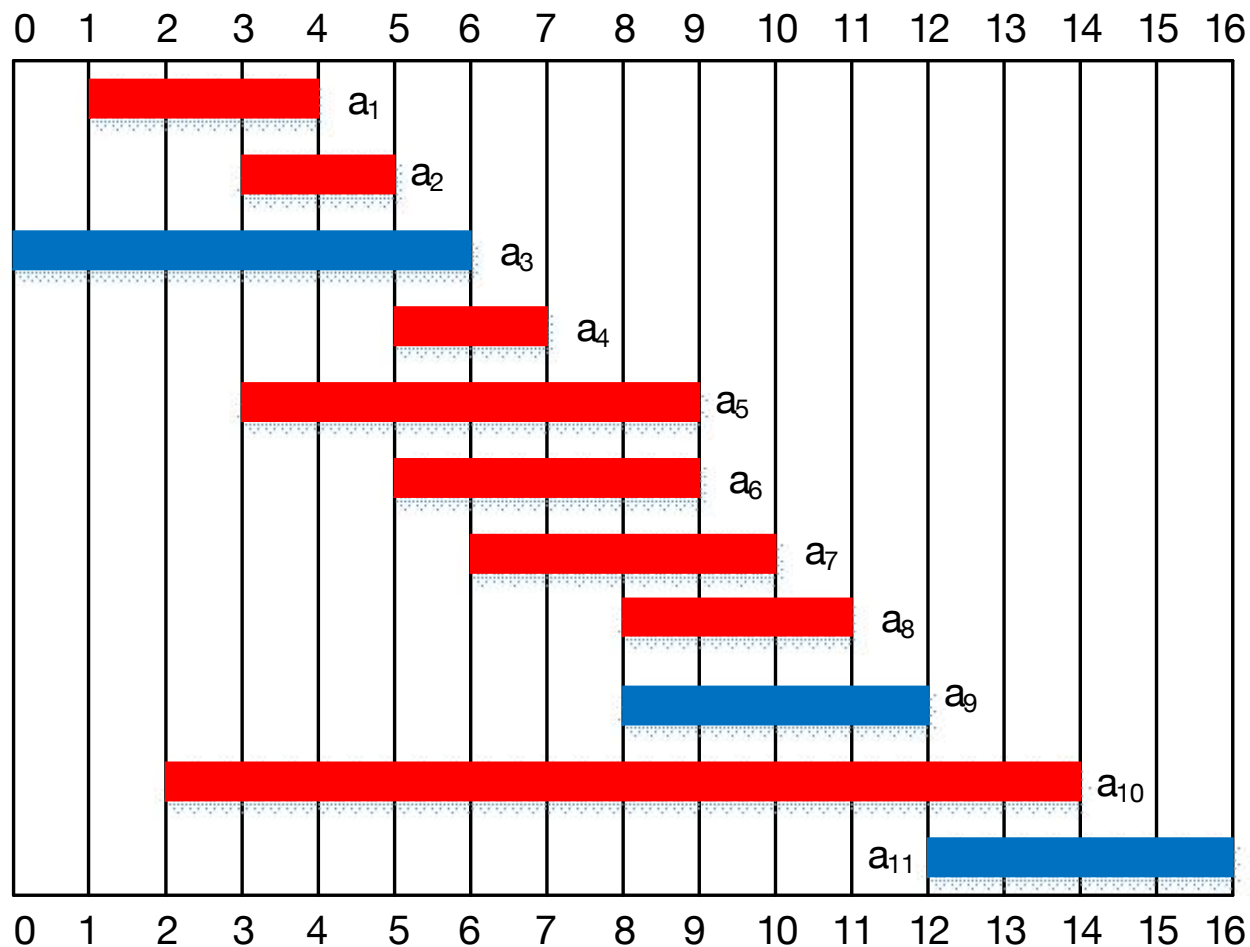
假设我们存在这样一个活动集合 $S = \{a_1, a_2, a_3, a_4, \dots, a_n\}$, 其中每一个活动 a_i 都有一个开始时间 s_i 和结束时间 f_i 保证 $(0 \leq s_i < f_i)$, 活动 a_i 进行时, 那么它占用的时间为 $[s_i, f_i)$, 现在这些活动占用一个共同的资源(教室), 就是这些活动会在某一时间段里面进行安排, 如果两个活动 a_i 和 a_j 的占用时间 $[s_i, f_i), [s_j, f_j)$ 不重叠, 那么就说明这两个活动是兼容的, 也就是说当 $s_j \geq f_i$ 或者 $s_i \geq f_j$ 那么活动 a_i, a_j 是兼容的。在活动选择问题中, 我们希望选出一个最大兼容活动集, 即在同一间教室能安排数量最多的活动。

活动选择问题



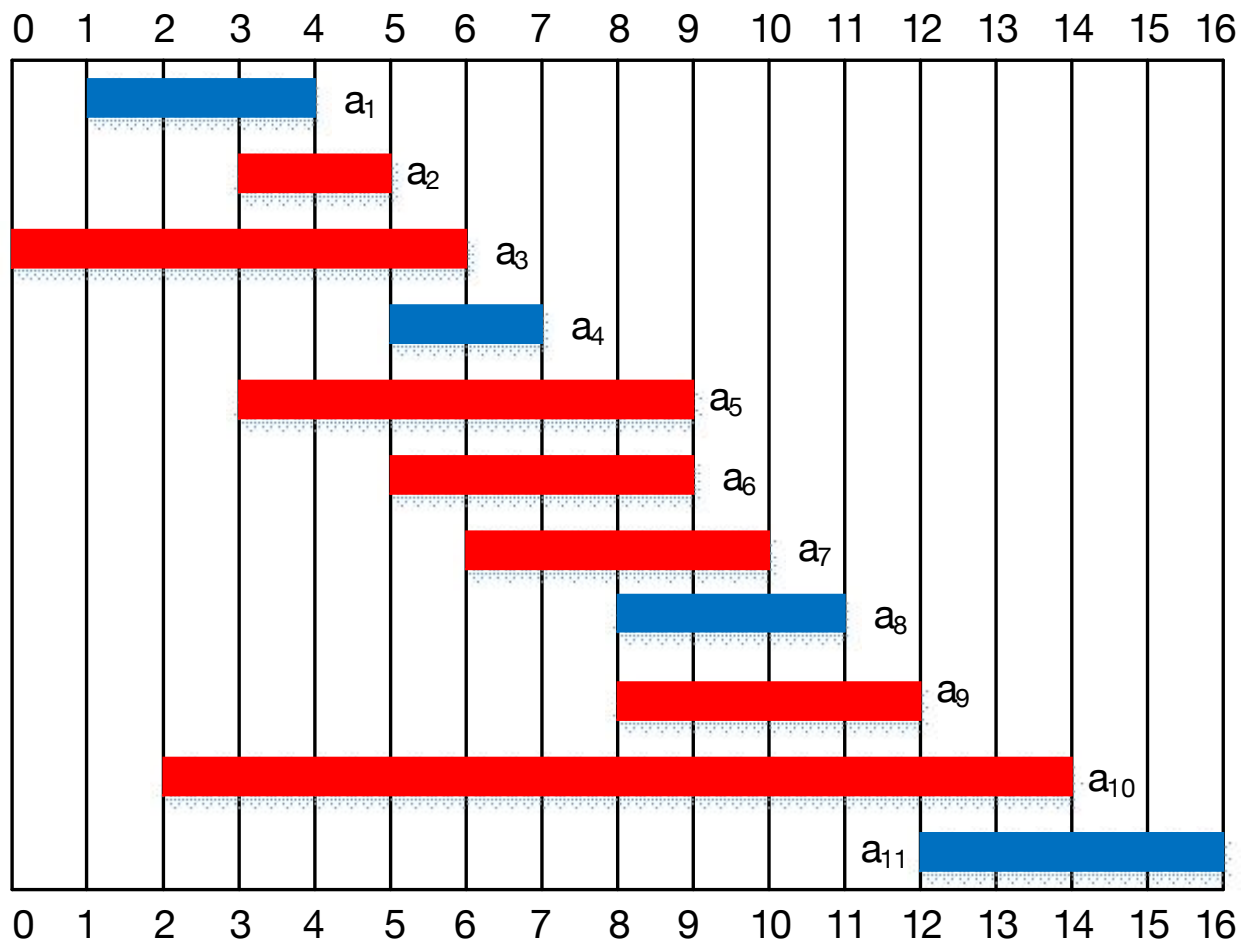
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题



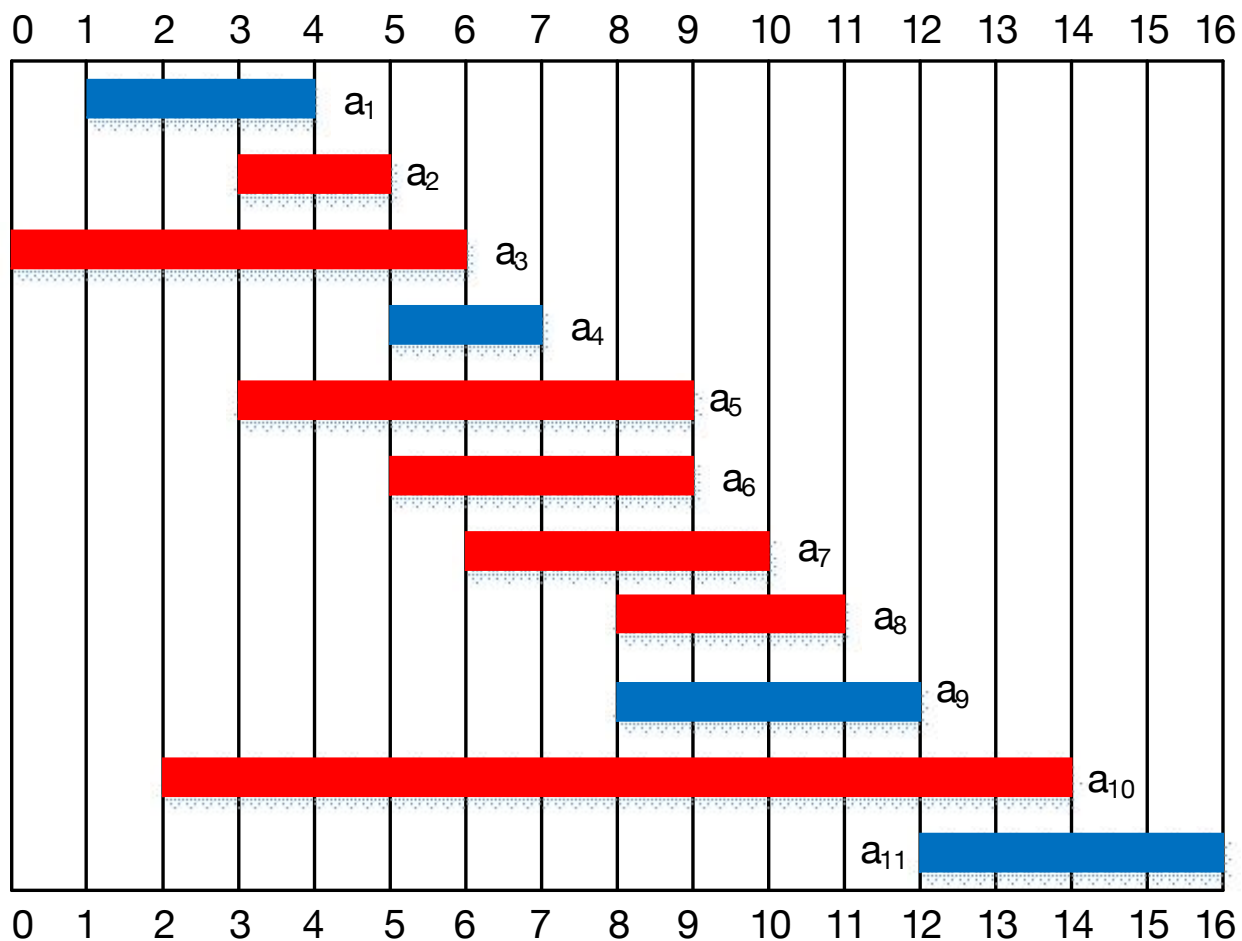
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题



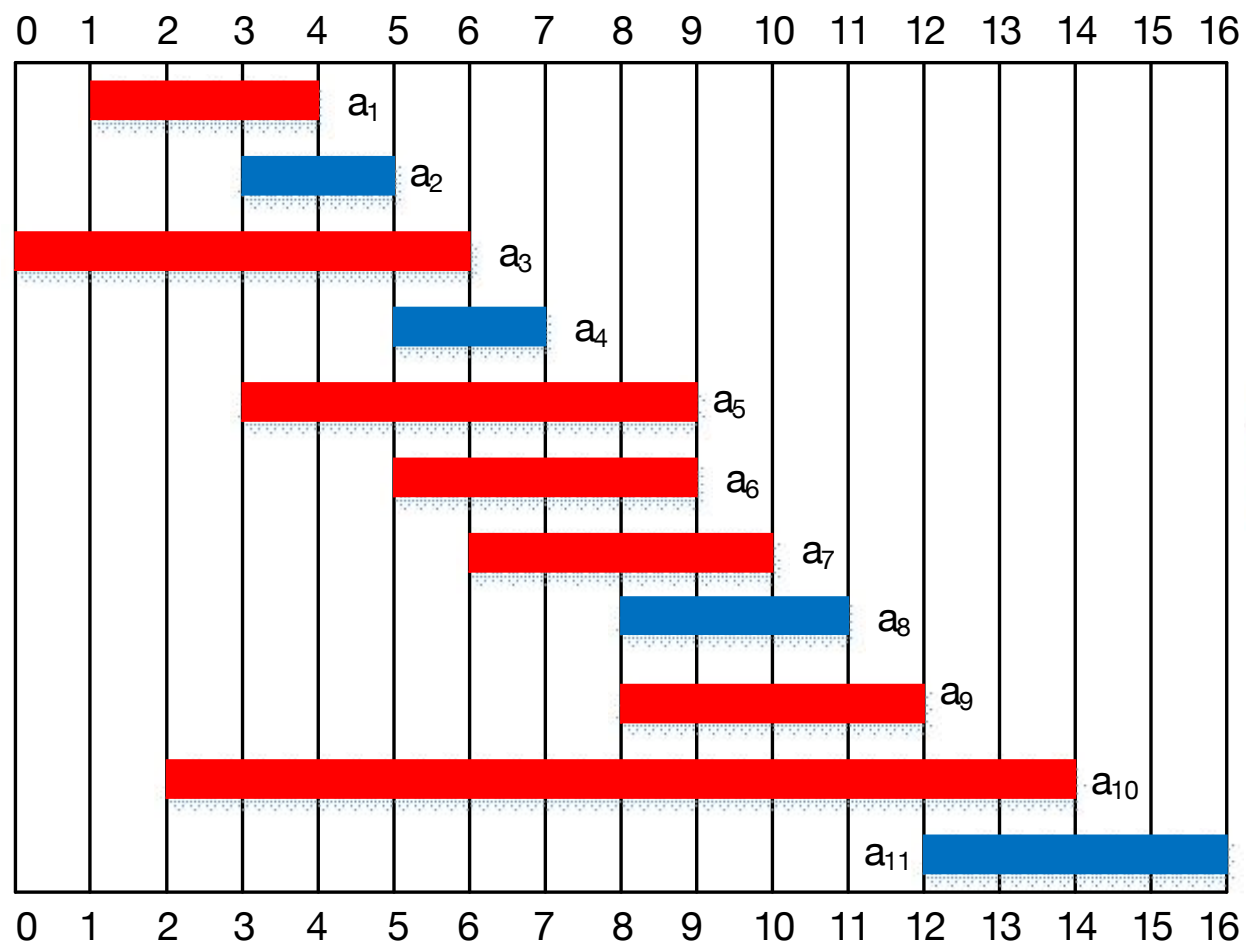
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题



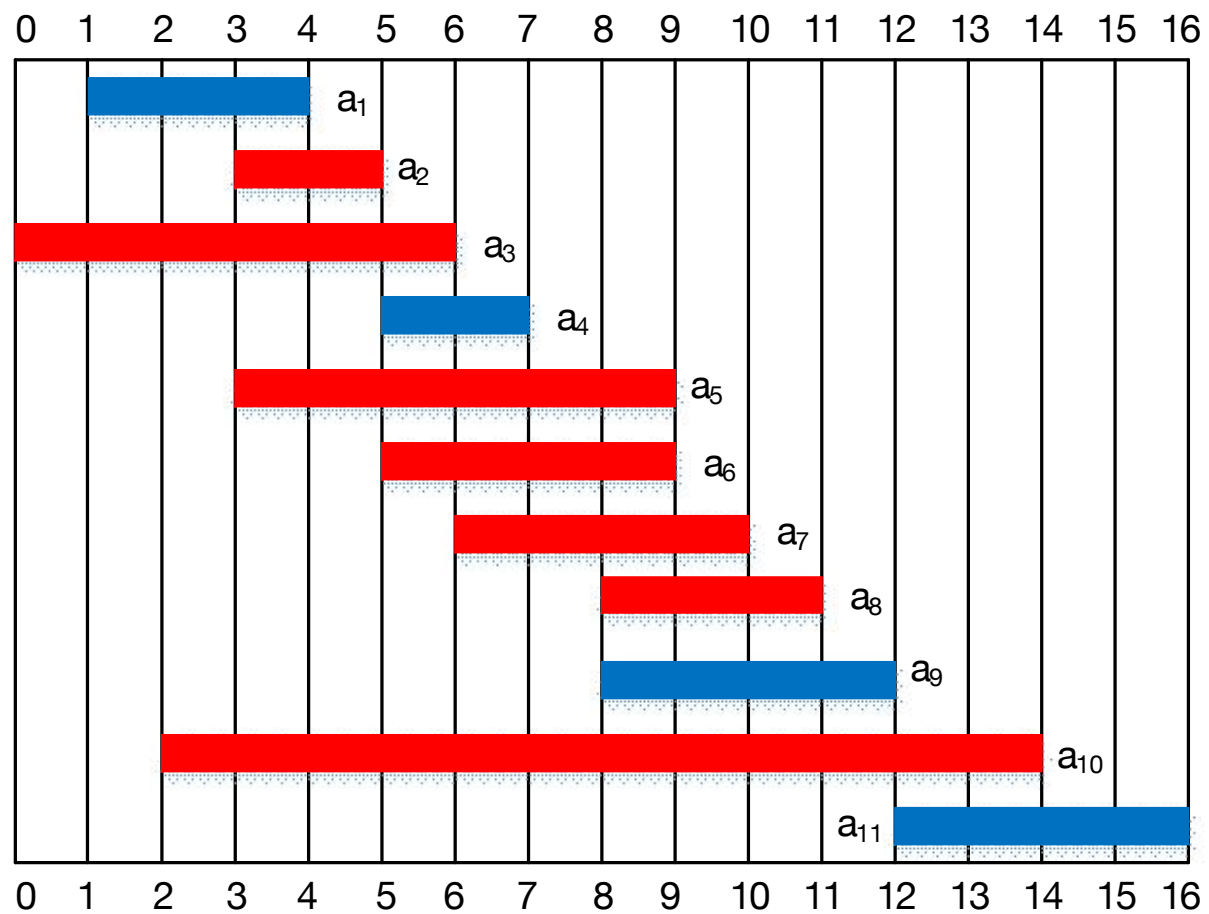
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题



i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题



i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

活动选择问题-最优子结构



$$\overbrace{a_1, a_2, \dots, a_i, \underbrace{a_{i+1}, \dots, a_{j-1}}_{S_{ij}}, a_j, \dots, a_{n-1}, a_n}^S$$

假设在有活动集合 S_{ij} 且其最大兼容子集为 A_{ij} , A_{ij} 之中包含活动 a_k

令 $A_{ik} = A_{ij} \cap S_{ik}$ 和 $A_{kj} = A_{ij} \cap S_{kj}$

$$A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$$

S_{ij} 里面的最大活动兼容子集个数为

$$|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$$

活动选择问题-最优子结构



最优子结构证明:

通过一个活动 a_k 将问题 S_{ij} 分成两个子问题: S_{ik} 和 S_{kj}

S_{ij} 的最优解 A_{ij} 必然包含其子问题 S_{ik} 和 S_{kj} 的最优解: A_{ik} 和 A_{kj} 。

活动选择问题-最优子结构



递归式:

$$c[i, j] = c[i, k] + c[k, j] + 1$$

$$c[i, j] = \begin{cases} 0 & i = j - 1 \\ \max\{c[i, k] + c[k, j] + 1\} (i \leq k \leq j) & i \geq j \end{cases}$$

$$c[i, j]$$
[illegible]

活动选择问题-动态规划法



这里的 $S(i,j)$ 一共大约有 N^2 个选择
复杂度是 $O(N^3)$

活动选择问题-贪心算法



活动选择问题-贪心策略

输入: $S = \{1, 2, \dots, n\}$ 为 n 项活动的集合, s_i, f_i 分别为活动 i 的开始和结束时间, 活动 i 与 j 相容 $\Leftrightarrow s_i \geq f_j$ 或 $s_j \geq f_i$.

求: 最大的两两相容的活动集 A

实例	i	1	2	3	4	5	6	7	8	9	10	11
	s_i	1	3	0	5	3	5	6	8	8	2	12
	f_i	4	5	6	7	9	9	10	11	12	14	16

策略1: 排序使得 $s_1 \leq s_2 \leq \dots \leq s_n$, 从前向后挑选

策略2: 排序使得 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$, 从前向后挑选

策略3: 排序使得 $f_1 \leq f_2 \leq \dots \leq f_n$, 从前向后挑选

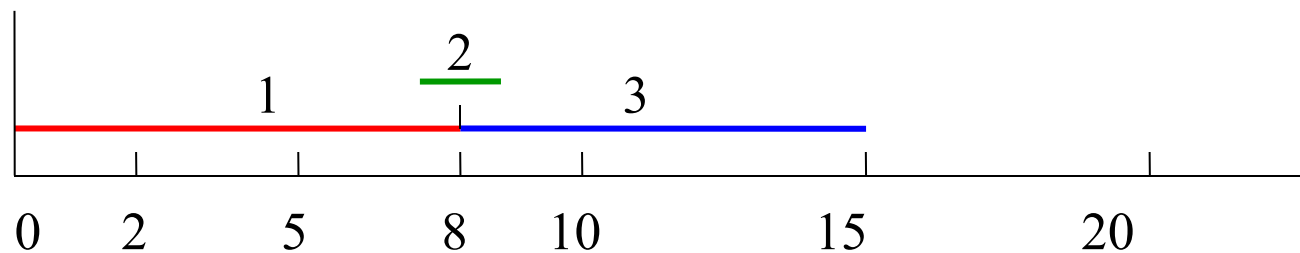
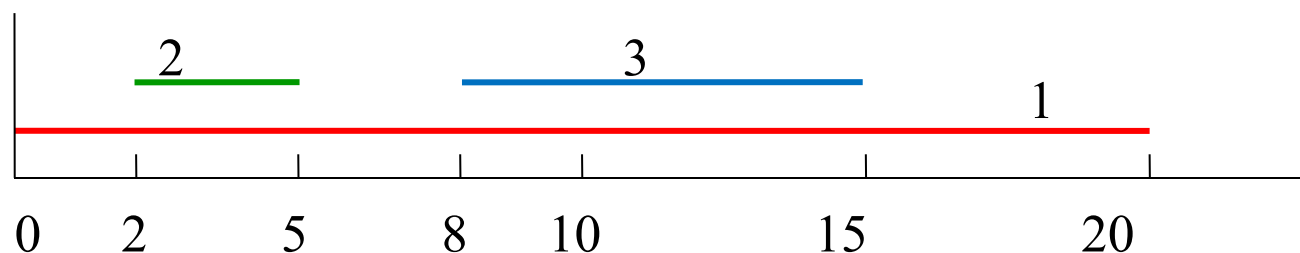
以上策略中的挑选都要注意满足相容性条件

活动选择问题-贪心算法



策略1: $S=\{1,2,3\}$, $s_1=0, f_1=20, s_2=2, f_2=5, s_3=8, f_3=15$

策略2: $S=\{1,2,3\}$, $s_1=0, f_1=8, s_2=7, f_2=9, s_3=8, f_3=15$



活动选择问题-贪心算法



贪心算法选择策略:

选择最早结束的活动。直观上认为每次选择最早结束的活动，后面的活动可选的时间就越充裕。

活动选择问题-贪心算法



我们的直觉是正确的吗?贪心选择——最早结束的活动——总是最优解的一部分吗?

定理16.1 考虑任意非空子问题 S_k , 令 a_m 是 S_k 中结束时间最早的活动, 则 a_m 在 S_k 的某个最大兼容活动子集中

证明: 假设 A_k 是 S_k 的一个最大兼容活动子集, 并且 a_j 为 A_k 中, 结束时间最早的活动。如果 $a_j=a_m$,则已得证;

如果 $a_m \neq a_j$, 则令集合 $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$ 。也就是将 A_k 中的元素 a_j 替换成 a_m 。因为 a_m 为 S_k 中结束时间最早的活动, 所以 A'_k 中的活动也是相互兼容的, 又因为 $|A_k| = |A'_k|$, 所以 A'_k 也是 S_k 的最大兼容活动子集, 所以得证。

活动选择问题-贪心算法



贪心算法选择策略3: 选择最早结束活动, 递归

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

- 1 $m = k+1$
- 2 **while** $m \leq n$ and $s[m] < f[k]$ // find the first activity in S_k to finish
- 3 $m = m+1$
- 4 **if** $m \leq n$
- 5 **return** $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$
- 6 **else return** $\emptyset$

活动选择问题-贪心算法



贪心算法选择策略3: 选择最早结束活动, 递归

GREEDY-ACTIVITY-SELECTOR(s, f)

```
1   $n = s.length$ 
2   $A = \{a_1\}$ 
3   $k = 1$ 
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A = A \cup \{a_m\}$ 
7           $k = m$ 
8  return  $A$ 
```


活动选择问题-贪心算法



求解活动选择问题总结

动态规划算法

- 1、动态规划算法通常以自底向上的方式解各子问题，全局最优解中一定包含某个局部最优解，但不一定包含前一个局部最优解，因此需要记录之前的所有最优解
- 2、动态规划的关键是状态转移方程，即如何由以求出的局部最优解来推导全局最优解
- 3、边界条件：即最简单的，可以直接得出的局部最优解。

贪心算法

- 1、而贪心算法则通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每做一次贪心选择就将所求问题简化为规模更小的子问题。
- 2、贪心算法中，作出的每步贪心决策都无法改变，因为贪心策略是由上一步的最优解推导下一步的最优解，而上一步之前的最优解则不作保留。

目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度
问题

第六节

找零问题

第七节

贪心算法与人工
智能算法



设计贪心算法经过下面几个步骤:

- ① 确定问题的最优子结构
- ② 基于问题的最优子结构设计一个递归算法
- ③ 证明我们做出的贪心选择, 只剩下一个子问题
- ④ 证明贪心选择总是安全的
- ⑤ 递归算法实现贪心策略
- ⑥ 将贪心算法转化为迭代算法



贪心算法的设计步骤简化:

- ① 将最优化问题简化为这样的形式：最初一个选择以后，只剩下一个子问题需要求解!
- ② 证明在做出贪心选择以后，原问题总是存在最优解，即贪心选择总是安全的!
- ③ 证明在做出贪心选择以后，剩下的子问题满足性质：其最优解与做出选择的组合在一起得到原问题的最优解，即最优子结构。



贪心算法的两个重要的性质:

- 贪心选择性质
- 最优子结构性质



贪心 vs 动态规划

➤ 01背包问题

一个正在抢劫商店的小偷发现了 n 个商品，第 i 个商品价值 v_i 元，重 w_i 磅， v_i 和 w_i 都是整数。小偷希望拿走价值尽可能高的商品，但是背包最多只能容纳 W 磅。他应该拿走哪些商品呢？

➤ 分数背包问题

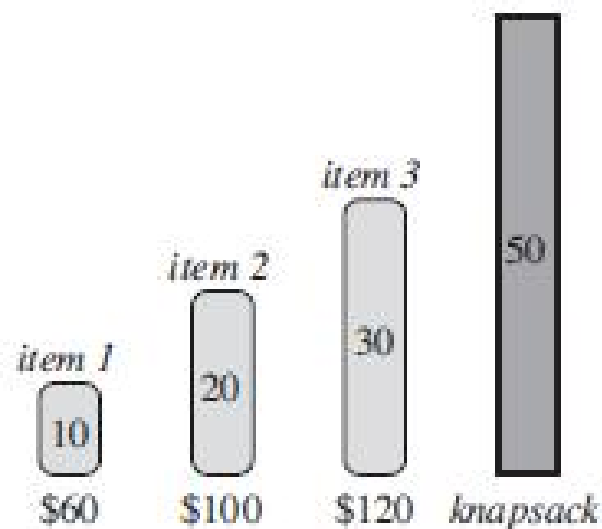
设定与0-1背包一样，但是对每个商品，小偷可以只拿走其一部分，而不是只能做出二元(0-1)选择。可以把0-1背包问题中的商品看做品质不一大小不同的金块，而分数背包问题中的商品更像是金砂。

贪心算法原理

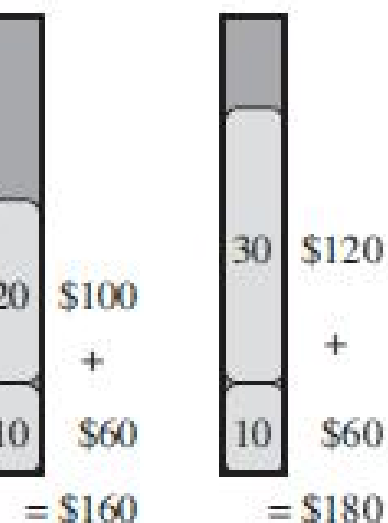
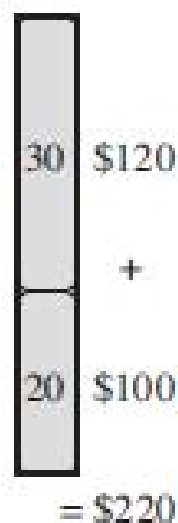


贪心 vs 动态规划

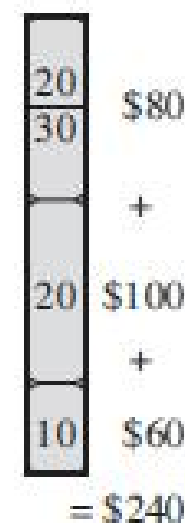
01背包问题/分数背包问题



(a)



(b)



(c)

一个实例，说明贪心策略对0-1背包问题无效。（a）小偷必须选择所示三个商品的一个子集总重量不超过50磅。（b）最优子集由商品2和商品3组成。虽然商品1有最大的每磅价值，但包含它的任何解都是次优的。（c）对于分数背包问题，按每磅价值降序拿走商品会生成一个最优解。



目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度问题

第六节

找零问题

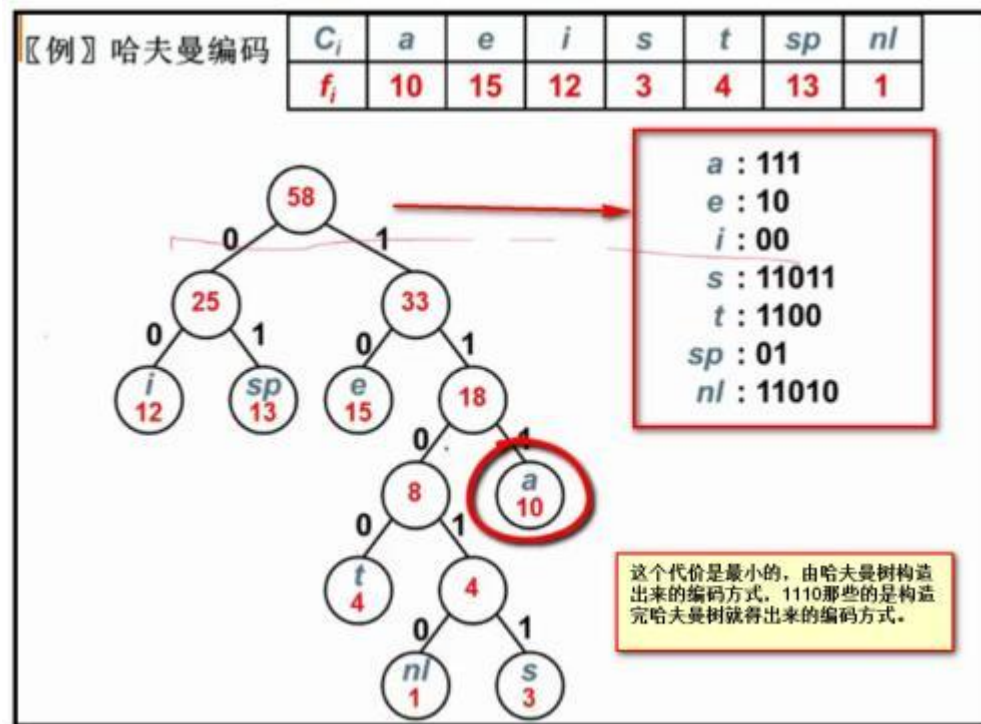
第七节

贪心算法与人工智能算法

哈夫曼编码



一种字符编码方式，常用于数据文件压缩。压缩率通常在20%~90%。



采取可变长编码方式，对文件中出现次数多的字符采取比较短的编码，对于出现次数少的字符采取比较长的编码，可以有效地减小总的编码长度。

哈夫曼编码



例子:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100



前缀码:

设 $Q = \{a_1, a_2, \dots, a_m\}$ 是一个0-1序列集合。如果 Q 中没有一个序列是另一个序列的前缀, 则称 Q 为前缀码。

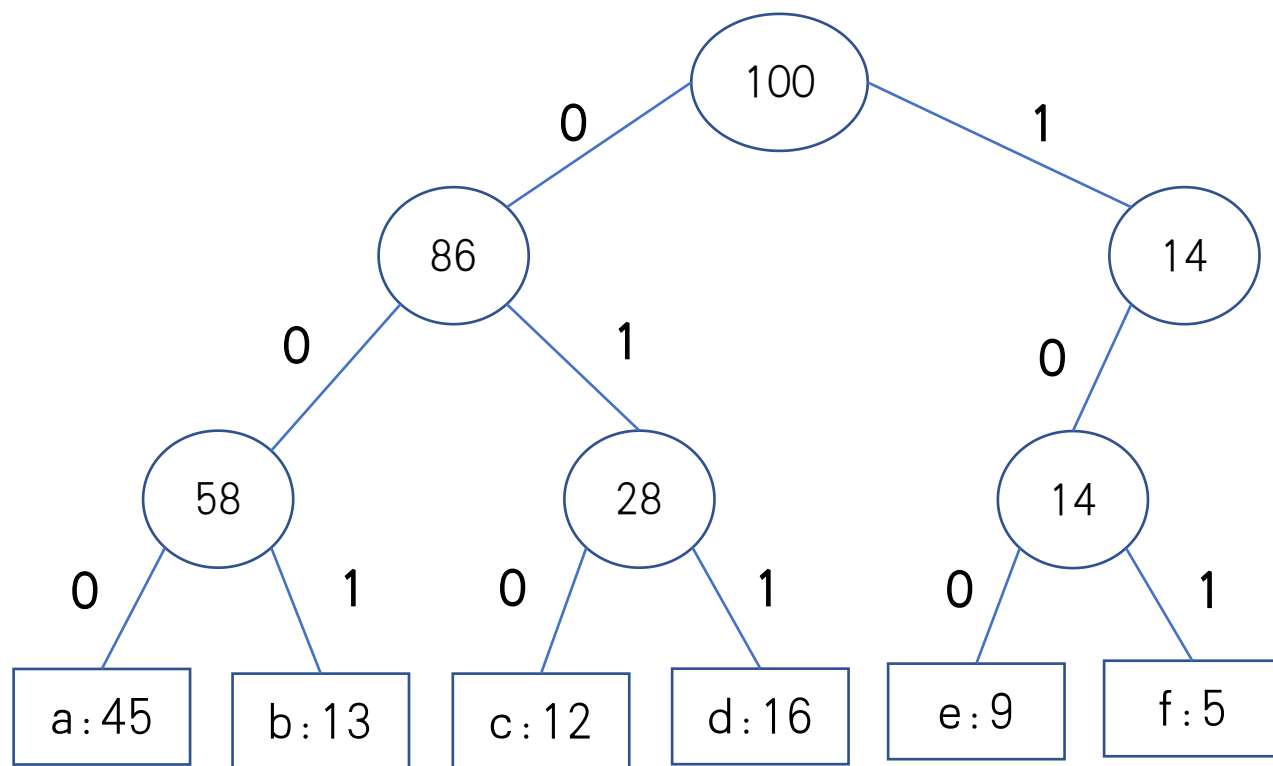
是指对字符集进行编码时, 要求字符集中任一字符的编码都不是其它字符的编码的前缀。

前缀码的作用是简化解码的过程。由于没有码字是其他码字的前缀, 编码文件的开始码字是无歧义的, 我们可以简单地识别出开始码字, 将其转换回原字符, 然后对编码文件剩余部分重复这种解码过程。

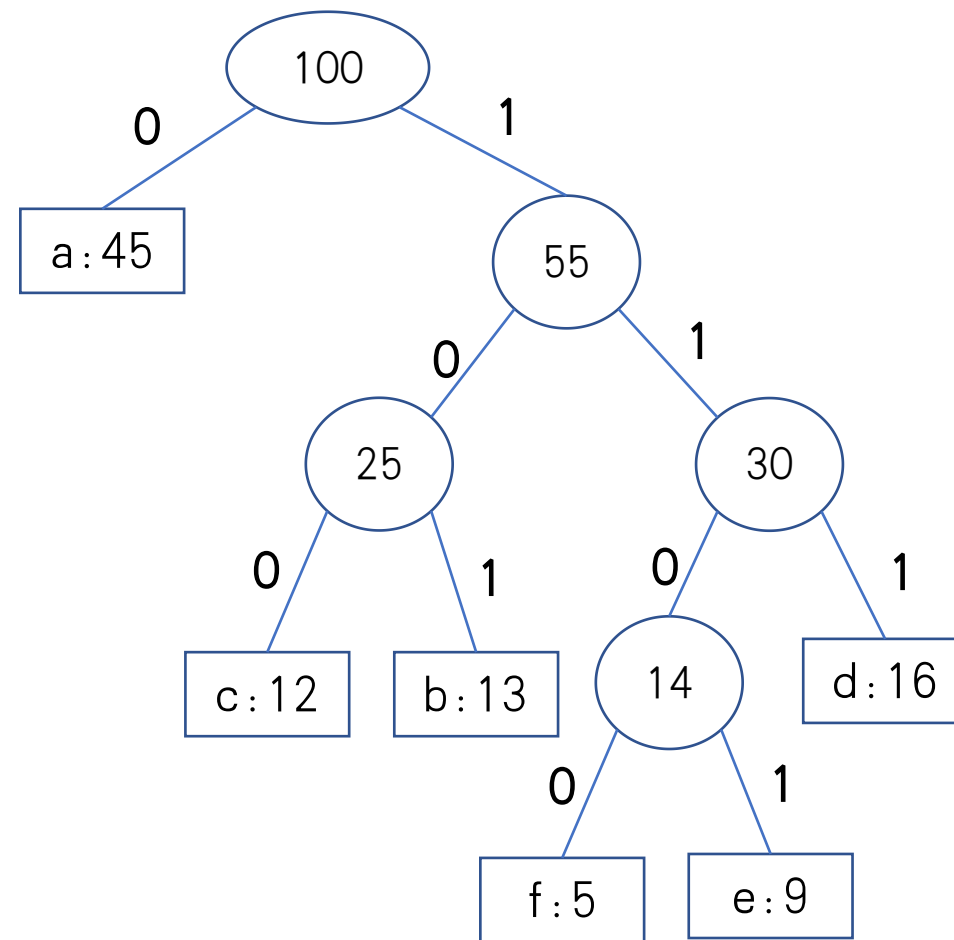
哈夫曼编码



编码的解码过程:



(a)



(b)



计算编码文件所需要二进制位

给定一颗对应前缀编码的树 T ，我们可以容易地计算出编码一个文件需要多少个二进制位。对于字母表 C 中的每个字符 c ，令属性 $c.freq$ 表示 c 在文件中出现的频率，令 $d_T(c)$ 表示 c 的叶结点在树中的深度。注意， $d_T(c)$ 也是字符 c 的码字的长度。则编码文件需要

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

个二进制位，我们将 $B(T)$ 定义为 T 的代价。

代价 $B(T)$ 最小的最优编码树，就是哈夫曼树。

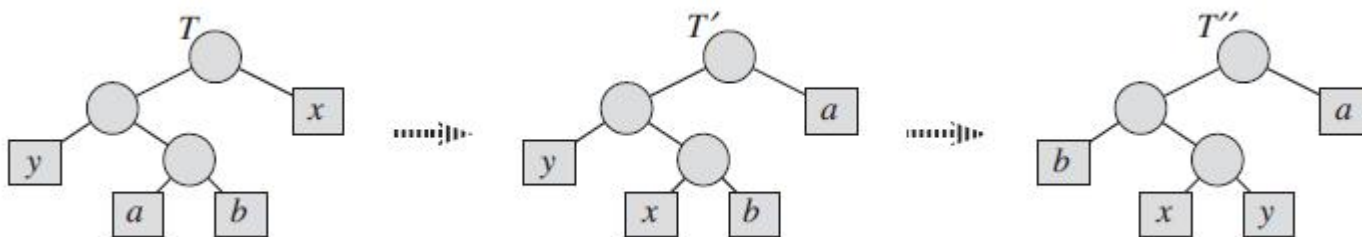
哈夫曼编码



算法的正确性证明

引理16.2 令 C 为一个字母表，其中每个字符 $c \in C$ 都有一个频率 $c.freq$ 。令 x 和 y 是 C 中频率最低的两个字符。那么存在 C 的一个最优前缀码， x 和 y 的码字长度相同，且只有最后一个二进制位不同。

证明：反证法。如果是 a 和 b 作为最优树 T 的最底层而不是 x 和 y 在最优树 T 的最底层，那么我们可以重新构造一个 T'' 满足最优树



哈夫曼编码



算法的正确性证明

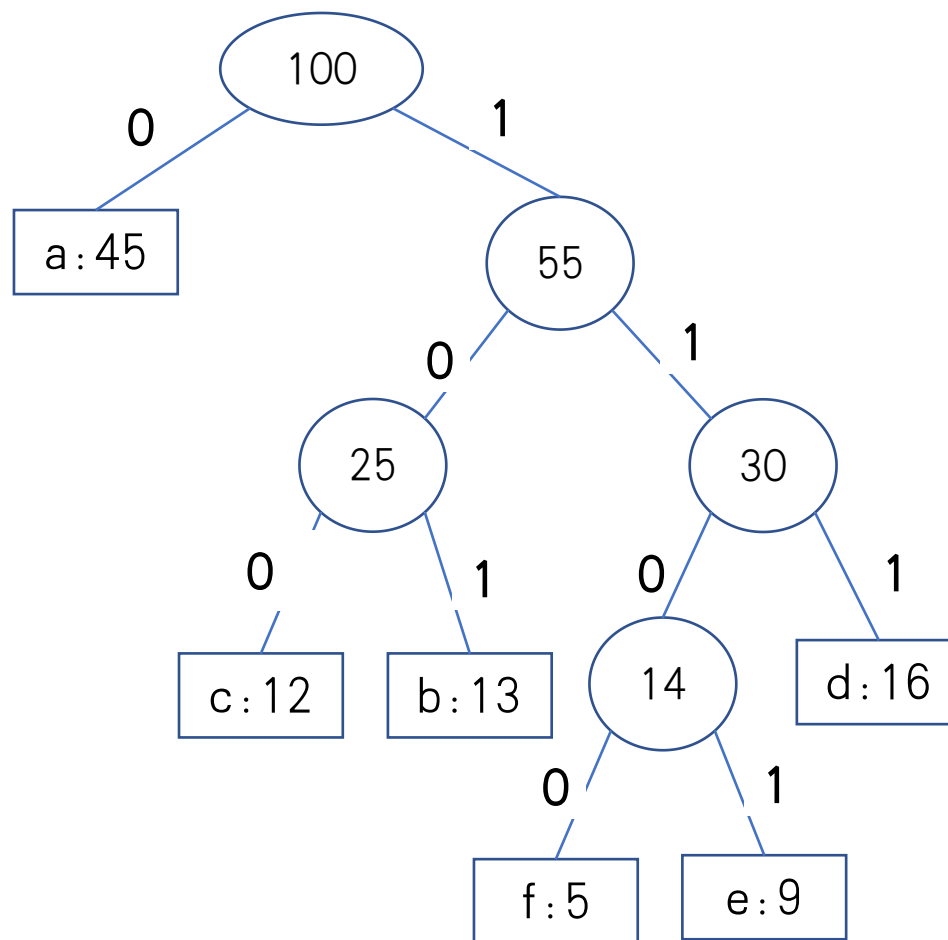
引理16.3 令 C 为一个给定的字母表，其中每个字符 $c \in C$ 都定义了一个频率 $c.\text{freq}$ 。令 x 和 y 是 C 中频率最低的两个字符。令 C' 为 C 去掉字符 x 和 y ,加入一个新字符 z 后得到的字母表，即 $C' = C - \{x, y\} \cup \{z\}$ 。类似 C ，也为 C' 定义 freq ，不同之处只是 $z.\text{freq} = x.\text{freq} + y.\text{freq}$ 。令 T' 为字母表 C' 的任意一个最优前缀码对应的编码树。于是我们可以将 T' 中叶结点 z 替换为一个以 x 和 y 为孩子的内部结点，得到树 T ,而 T 表示字母表 C 的一个最优前缀码。

证明：反证法。如果 T 不是最优编码树，那么有新的编码树 T'' ，他所对应的 T'' 比 T' 代价要小。

哈夫曼编码



哈夫曼树的应用：哈夫曼编码



哈夫曼编码



构造哈夫曼树算法的执行过程(贪心策略)

(a)

f:5

e:9

c:12

b:13

d:16

a:45

(b)

c:12

b:13

14

0

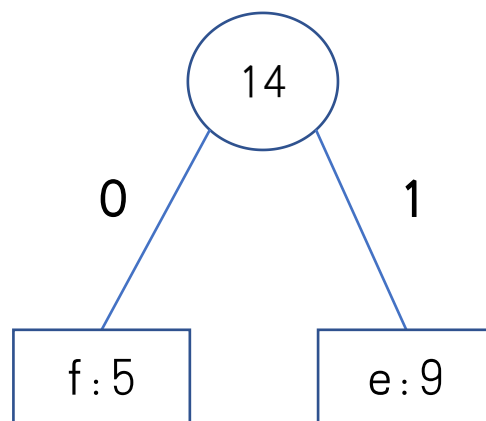
1

f:5

e:9

d:16

a:45

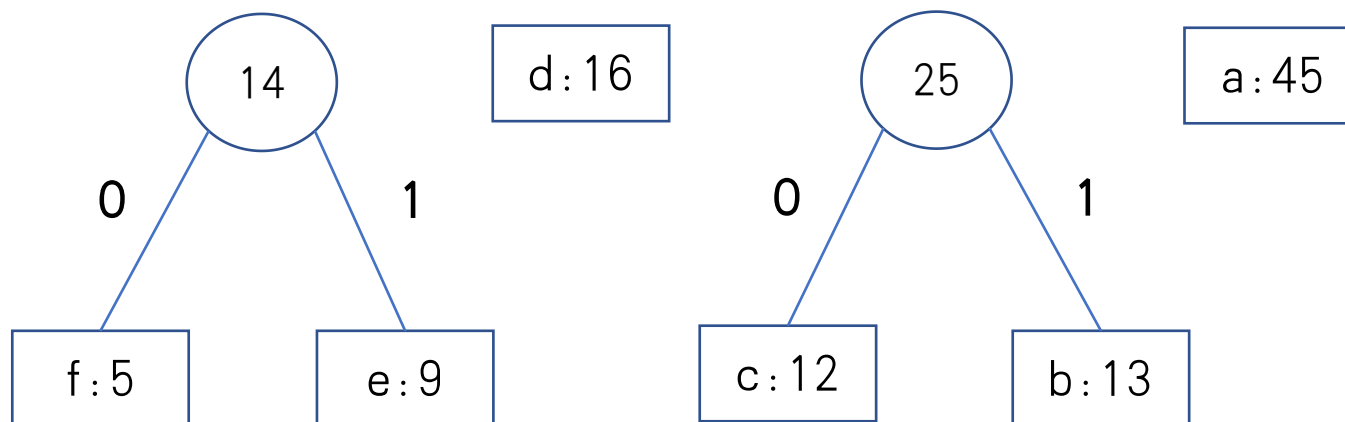


哈夫曼编码



构造哈夫曼树算法的执行过程(贪心策略)

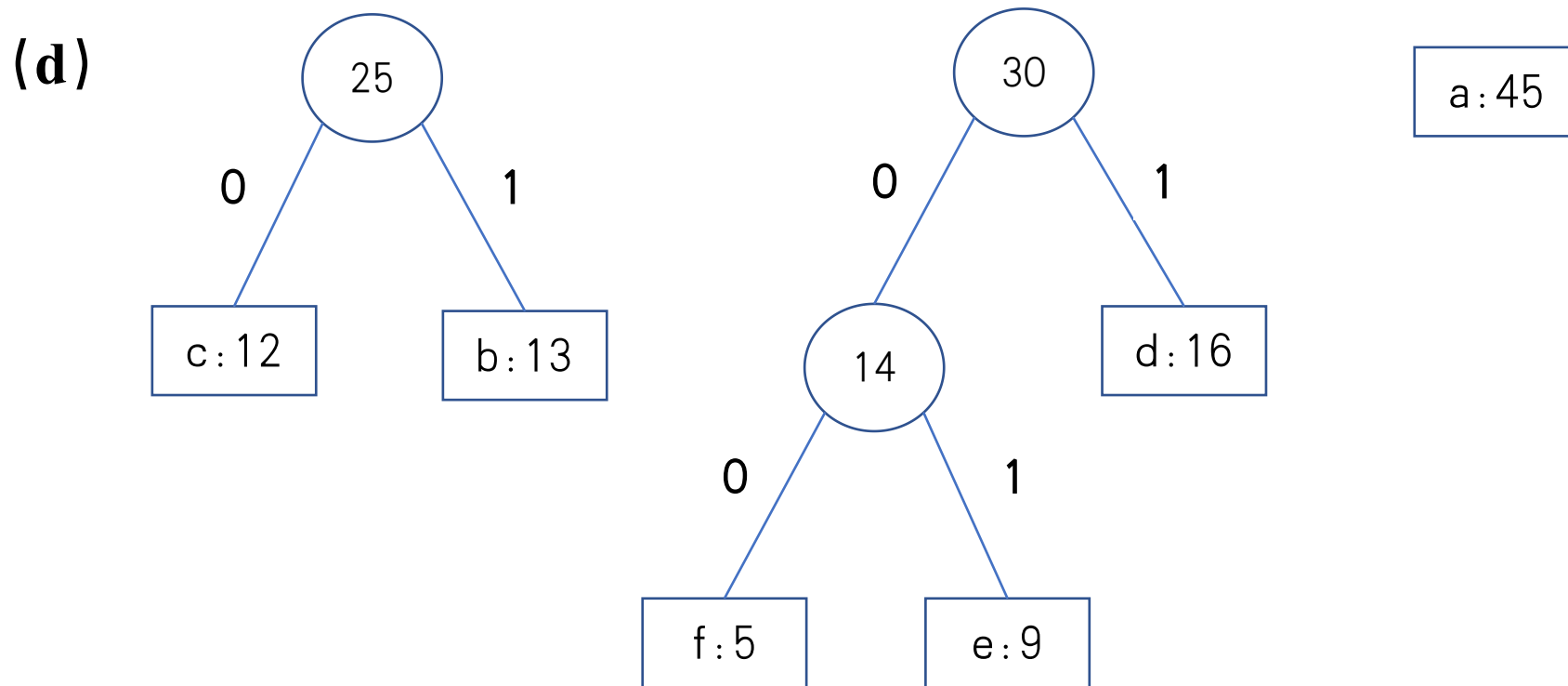
(c)



哈夫曼编码



构造哈夫曼树算法的执行过程(贪心策略)

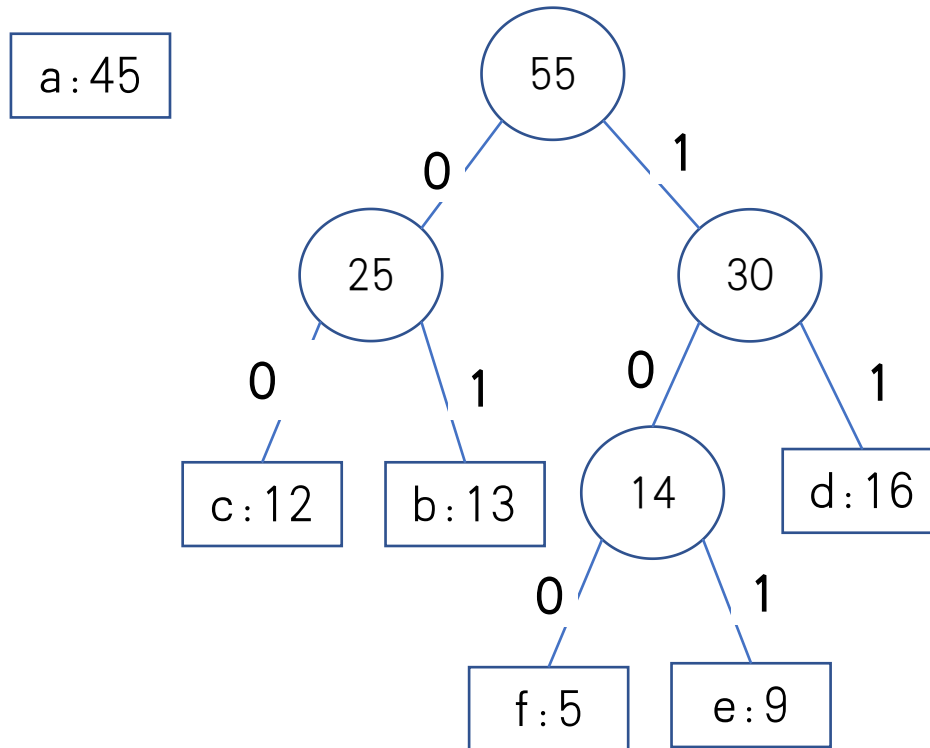


哈夫曼编码



构造哈夫曼树算法的执行过程(贪心策略)

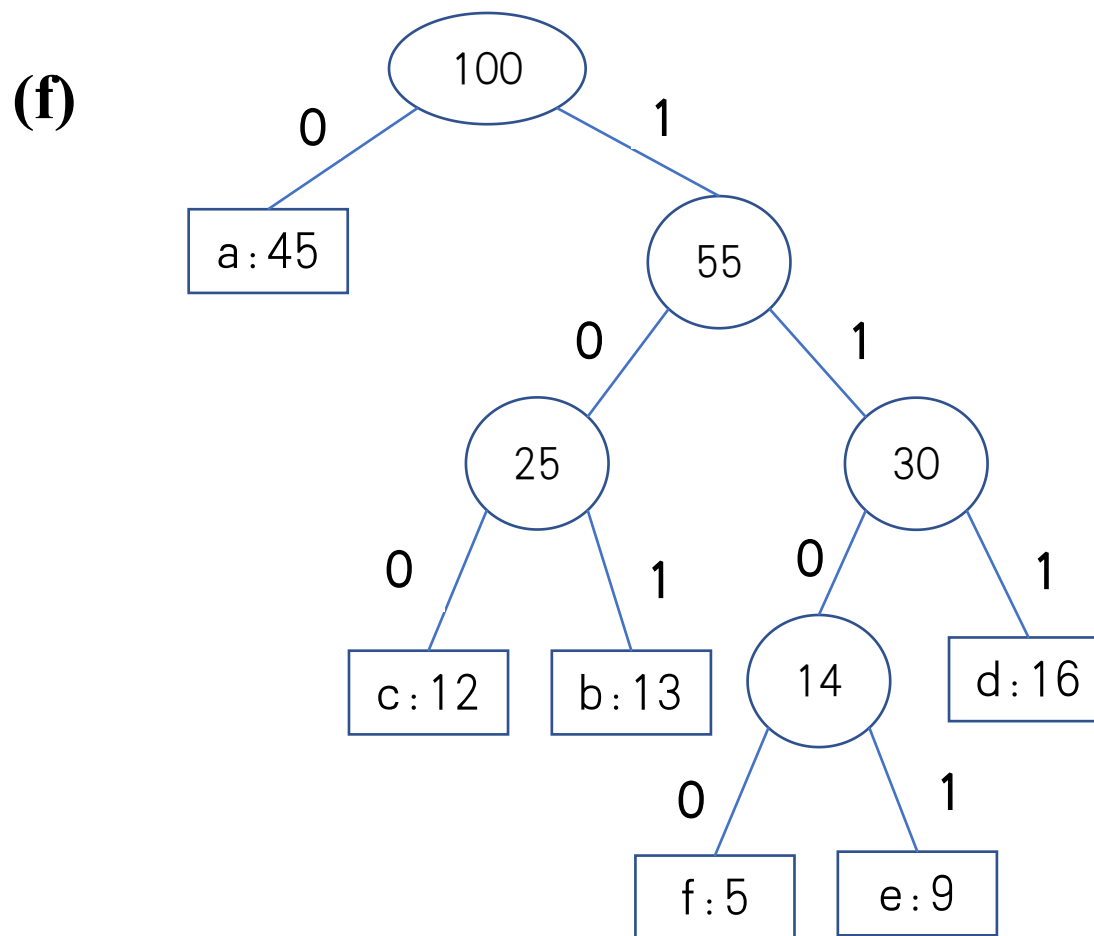
(e)



哈夫曼编码



构造哈夫曼树算法的执行过程(贪心策略)



哈夫曼编码



算法的正确性证明

引理16.4 哈夫曼编码可以由贪心得解

证明：根据引理16.2和16.3可得

目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度
问题

第六节

找零问题

第七节

贪心算法与人工
智能算法

最小延迟调度问题



问题定义：任务集合 S ， $\forall i \in S$ ， d_i 为截止时间， t_i 为加工时间， d_i, t_i 为正整数。一个调度 $f: S \rightarrow \mathbb{N}$ ， $f(i)$ 为任务 i 的开始时间。求最大延迟达到最小的调度，即求 f 使得

$$\min_f \{ \max_{i \in S} \{ f(i) + t_i - d_i \} \}$$

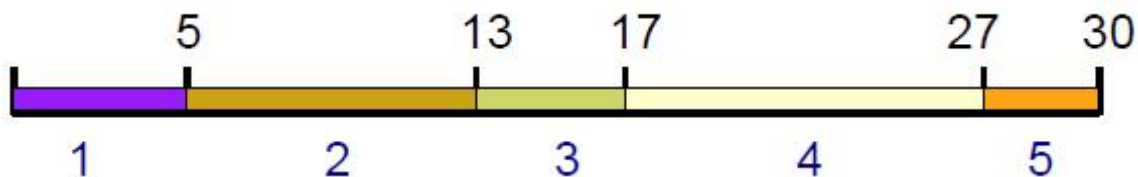
$$\forall i, j \in S, i \neq j, f(i) + t_i \leq t_j \vee f(i) + t_j \leq t_i$$

示例

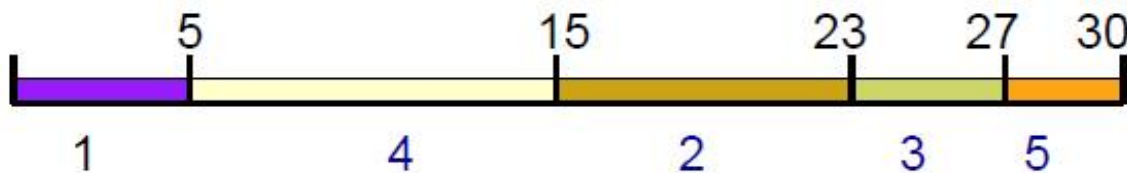


$S=\{1, 2, 3, 4, 5\}$, $d=<10, 12, 15, 11, 20>$, $t=<5, 8, 4, 10, 3>$

调度1: $f_1(1)=0, f_1(2)=5, f_1(3)=13, f_1(4)=17, f_1(5)=27$, 各任务延迟: 0, 1, 2, 16, 10; 最大延迟: 16



调度2: $f_2(1)=0, f_2(2)=15, f_2(3)=23, f_2(4)=5, f_2(5)=27$, 各任务延迟: 0, 11, 12, 4, 10; 最大延迟: 12



贪心策略选择



贪心策略1: 按照 t_i 从小到大安排任务

贪心策略2: 按照 $d_i - t_i$ 从小到大安排任务

贪心策略3: 按照 d_i 从小到大安排任务

策略1 对某些实例得不到最优解。反例: $t_1=1, d_1=100, t_2=10, d_2=10$

策略2 对某些实例得不到最优解。反例: $t_1=1, d_1=2, t_2=10, d_2=10$



算法思想：按照截止时间 d_i 从小到大选择任务，安排时不留空闲时间

算法伪代码：

Sort(S) 使得 $d_1 \leq d_2 \leq \dots \leq d_n$

$f(1)=0$, $i=2$

while $i \leq n$ do

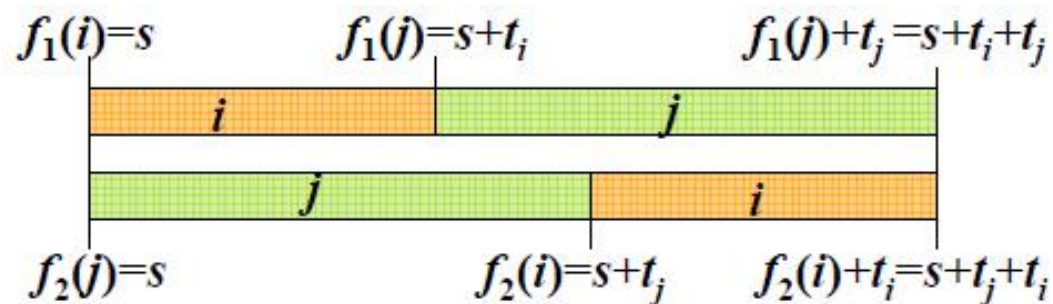
$f(i)=f(i-1)+t_{i-1}$; $i=i+1$

正确性证明



证明思路： 最优解中可以不存在相邻逆序任务。所谓相邻逆序任务就是 (i, j) : $f(i) < f(j)$ 且 $d_i > d_j$

证明： 如存在相邻逆序任务 (i, j) ，那么交换这个两个任务不增加延迟问题



i 在 f_2 结束时间= j 在 f_1 的结束时间= $s + t_i + t_j$;

i 在 f_2 的延迟: $L_{2i} = (s + t_i + t_j) - d_i$; j 在 f_1 的延迟: $L_{1j} = (s + t_i + t_j) - d_j$

目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度
问题

第六节

找零问题

第七节

贪心算法与人工
智能算法

找零问题



考虑用最少的硬币找 n 美分零钱的问题。假设每种硬币的面额都是整数。

a)设计贪心算法求解找零问题，假定有25美分、10美分、5美分和1美分4种面额的硬币。证明你的算法能找到最优解。

找零问题



贪心算法：先尽可能用25美分，然后尽可能用10美分，再尽可能用5美分，最后尽可能用1美分



找零问题



a)设计贪心算法求解找零问题，假定有25美分、10美分、5美分和1美分4种面额的硬币。证明你的算法能找到最优解。

证明思路：若 n 是正整数，则用25美分、10美分、5美分和1美分等尽可能少的硬币找出的 n 美分零钱中，至多有2个10美分、至多有1个5美分、至多有4个1美分硬币，而不能有2个10美分和1个5美分硬币。用10美分、5美分和1美分硬币找出的零钱不能超过24美分。

找零问题正确性证明



引理1: 若 n 是正整数, 则用25美分、10美分、5美分和1美分等尽可能少的硬币找出的 n 美分零钱中, 至多有2个10美分、至多有1个5美分、至多有4个1美分硬币, 而不能有2个10美分和1个5美分硬币。用10美分、5美分和1美分硬币找出的零钱不能超过24美分。

证明: 证用反证法。证明如果有超过规定数目的各种类型硬币, 就可以用等值的数目更少的硬币来替换。

- ① 如果有3个10美分硬币, 就可以换成1个25美分和1个5美分硬币;
- ② 如果有2个5美分硬币, 就可以换成1个10美分硬币
- ③ 如果有5个1美分硬币, 就可以换成1个5美分硬币;
- ④ 如果有2个10美分和1个5美分硬币, 就可以换成1个25美分硬币。

由于至多可以有2个10美分、1个5美分和4个1美分硬币, 而不能有2个10美分和1个5美分硬币, 所以当用尽可能少的硬币找 n 美分零钱时, 24美分就是用10美分、5美分和1美分硬币能找出最大值。

找零问题正确性证明



引理2: 找 n 美分零钱的最优方式中使用25美分硬币的个数 q' ，一定等于贪心算法所用25美分硬币的个数。

证明: 贪心算法使用尽可能多的25美分硬币，所以 $q' \leq q$ 。但是 q' 也不能小于 q 。假如 q' 小于 q ，需要在这种最优方式中用10美分、5美分和1美分硬币至少找出25美分零钱。而根据引理1，这是不可能的。

找零问题正确性证明



正确性证明： 找 n 美分零钱的贪心算法能得到最优解。

证明： 根据引理2，我们可以知道，找 n 美分零钱的贪心算法中25美分数量就是最优解中25美分数量。

由于在找 n 美分零钱的贪心解和最优解中一定有同样多的25美分硬币，所以在这两种方式中10美分、5美分和1美分硬币的总值一定相等，并且这些硬币的总值不超过24美分。10美分硬币的个数一定相等，因为贪心算法使用尽可能多的10美分硬币。而根据引理1，当使用尽可能少的硬币找零钱时，至多使用1个5分硬币和4个1分硬币，所以在找零钱的最优方式中也使用尽可能多的10美分硬币。类似地，5美分硬币的个数相等；最终，1美分的个数相等

找零问题



b)假定硬币的面额是 c 的幂, 即面额 $c^0, c^1, \dots, c^k$, c 和 k 为整数, $c > 1$, $k \geq 1$ 。

证明: 贪心算法总能得到最优解。

证明: 同a问, 由于 $c^0 + c^1 + \dots + c^{k-1} = c^k - 1 < c^k$, 故当 n 大于 c^k 时, 可以分解为 c^k 与 $n - c^k$ 的值, 其中 c^k 只用一个硬币值为 c^k 的硬币就能得到最少硬币数, 而子问题变成 $n - c^k$ 的最少硬币数, 依次类推, 贪心算法总能得到最好的结果。

找零问题



c)设计一组硬币面额，使得贪心算法不能保证得到最优解。这组硬币面额中应该包含1美分，使得对每个零钱值都存在找零方案。

证明：如果出现一组硬币6，5，1。当遇到10分时，按照贪心算法将分解为 $6+4*1$ ，而其实为 $2*5$ 。



找零问题



d)设计一个 $O(nk)$ 时间的找零算法，适用于任何 k 种不同面额的硬币，假定问题包含1美分硬币。

证明：假设 $m[n]$ 表示找零 n 美分需要的最少硬币数，硬币面值为 $c_0, c_1, \dots, c_k$ ，并令 $m[0]=0$ ，则 $m[n]=1$ 如果 n 等于某个 c_i ，否则 $m[n] = \min\{m[n - c_0] + 1, m[n - c_1] + 1, \dots, m[n - c_k] + 1\}$ 。基于此公式，可以设计动态规划算法。

目录

第一节

概述

第二节

活动选择问题

第三节

贪心算法原理

第四节

哈夫曼编码

第五节

最小延迟调度
问题

第六节

找零问题

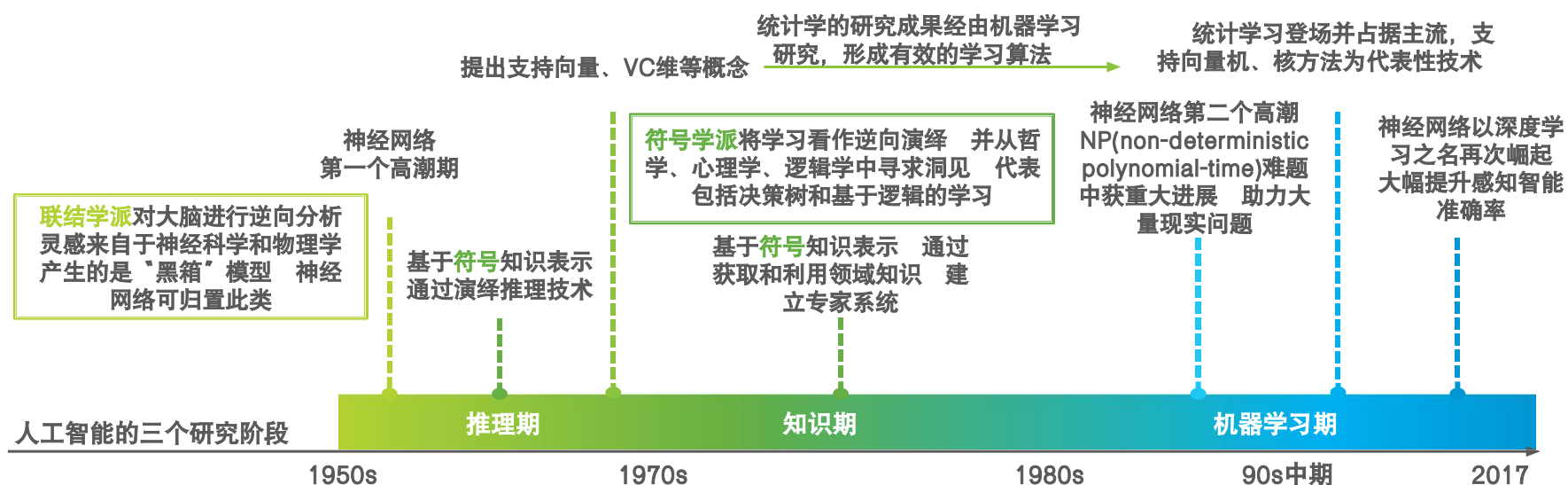
第七节

贪心算法与人工
智能算法

人工智能算法



- 从以“推理”为重点到以“知识”为重点，再到以“学习”为重点
- 机器可以自动“学习”的算法，即从数据中自动分析获得规律，并利用规律对未知数据进行预测的算法。目前，机器学习=“分类”
- 人工智能 > 机器学习 > 深度学习



机器学习算法与贪心算法关系



现有机器学习算法利用输入数据多轮迭代地生成模型，每轮迭代中都是贪心地选择当前最优模型来生成。

为了进一步降低计算空间进而提高效率，在贪心的基础上，很多机器学习算法在执行过程中还加上了随机的思想。它们随机地在下一步可能的最优模型中选择若干分支。

示例数据（西瓜数据集2.0）

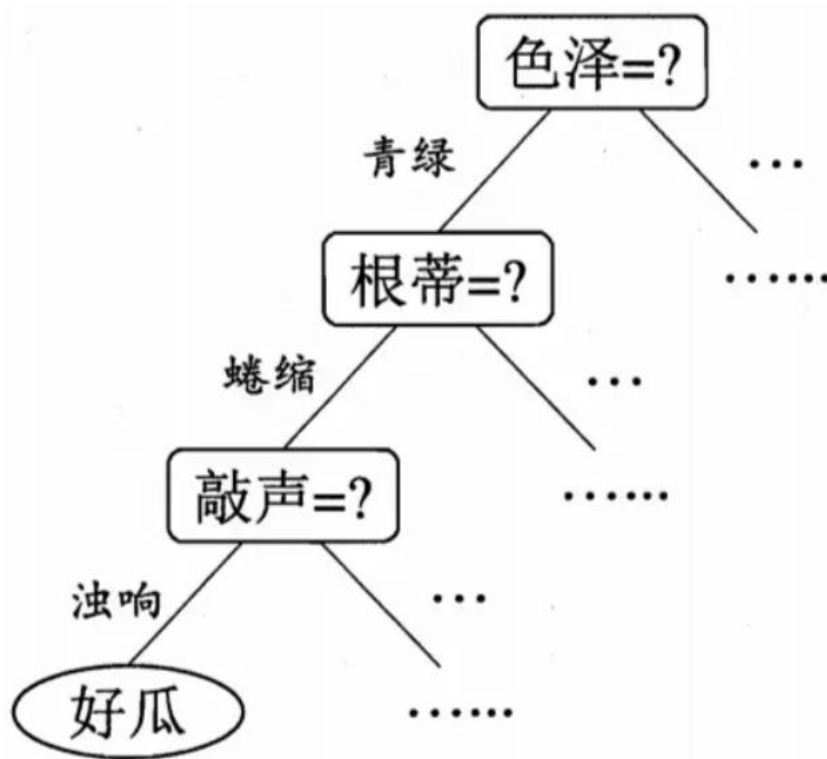


编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

机器学习典型算法：决策树



- 决策树是一种模仿人类决策过程的监督学习算法，其结构像一个倒置的树。它通过一系列**判断题**对数据进行层层划分，最终得出分类结果。



机器学习典型算法：决策树



□ 决策树由三种基本元素构成：

- **根节点：** 树的起点，包含了全部的数据样本，代表第一个需要判断的问题。
- **内部节点：** 代表一个特征或属性的判断（例如：“色泽=? ”）。从内部节点延伸出的分支代表判断的结果。
- **叶节点：** 树的终点，不再有分支。它代表了最终的决策结果，在分类任务中是类别标签（如“好瓜/坏瓜”）。

□ 从根节点到任意一个叶节点的路径，就构成了一条完整的决策规则。

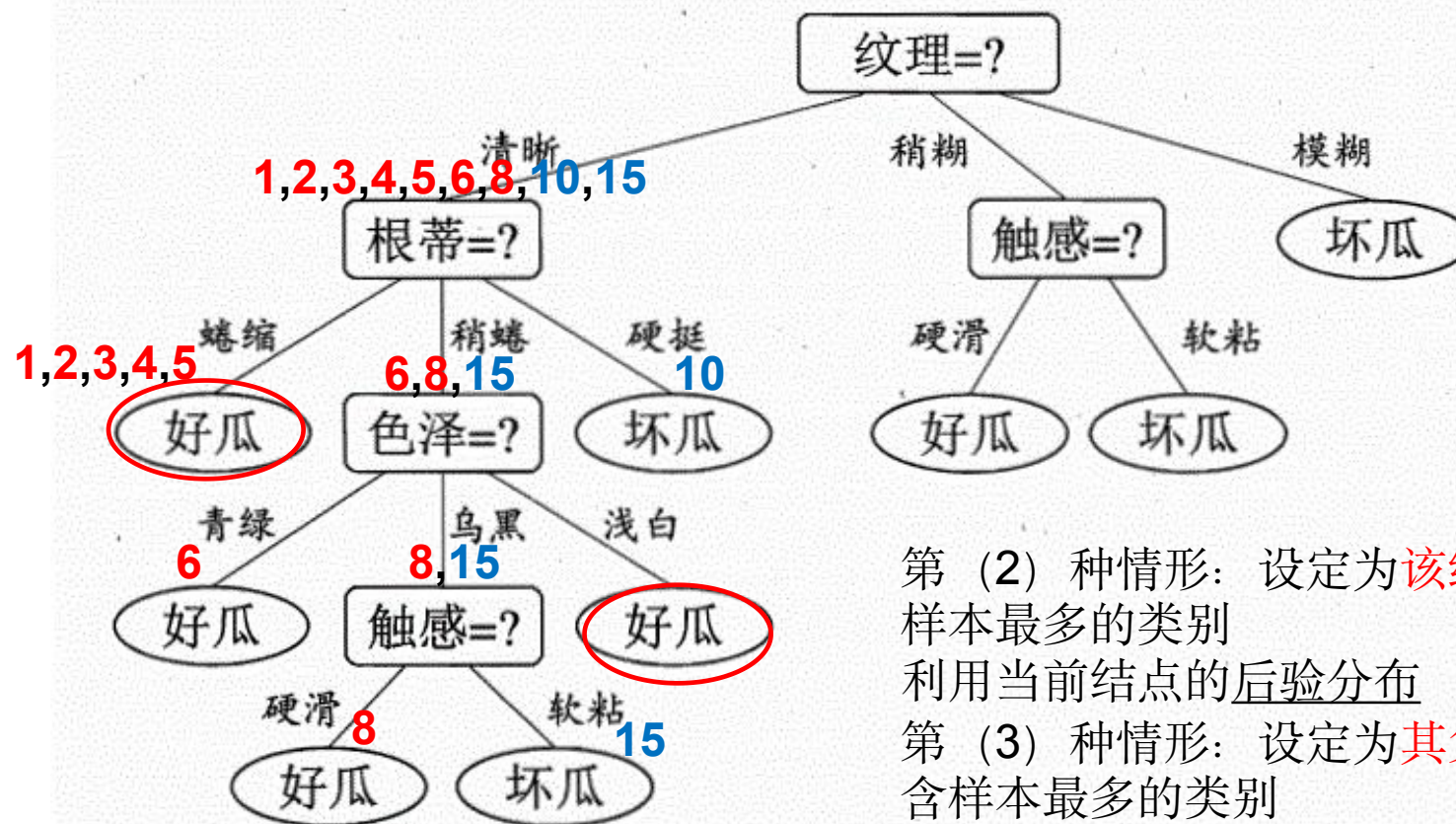
机器学习典型算法：决策树



一棵决策树的生成过程主要分为以下3个部分：

- 特征选择：特征选择是指从训练数据中众多的特征中选择一个特征作为当前节点的分裂标准，如何选择特征有着很多不同量化评估标准标准，从而衍生出不同的决策树算法。
- 决策树生成：根据选择的特征评估标准，从上至下递归地生成子节点，直到数据集不可分则停止决策树停止生长。树结构来说，递归结构是最容易理解的方式。
- 剪枝：决策树容易过拟合，一般来需要剪枝，缩小树结构规模、缓解过拟合。剪枝技术有预剪枝和后剪枝两种。

示例决策树



决策树典型算法



Hunt算法:

递归返回, 情形(1).

递归返回, 情形(2).

我们将在下一节讨论如何获得最优划分属性.

递归返回, 情形(3).

从 A 中去掉 a_* .

输入: 训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
属性集 $A = \{a_1, a_2, \dots, a_d\}$.
过程: 函数 TreeGenerate(D, A)

- 1: 生成结点 node;
- 2: if D 中样本全属于同一类别 C then
- 3: 将 node 标记为 C 类叶结点; return 无需划分
- 4: end if
- 5: if $A = \emptyset$ OR D 中样本在 A 上取值相同 then
- 6: 将 node 标记为叶结点, 其类别标记为 D 中样本数最多的类; return
- 7: end if
- 8: 从 A 中选择最优划分属性 a_* ; 无法划分
- 9: for a_* 的每一个值 a_*^v do
- 10: 为 node 生成一个分支; 令 D_v 表示 D 中在 a_* 上取值为 a_*^v 的样本子集;
- 11: if D_v 为空 then
- 12: 将分支结点标记为叶结点, 其类别标记为 D 中样本最多的类; return
- 13: else
- 14: 以 TreeGenerate($D_v, A \setminus \{a_*\}$) 为分支结点 不能划分
- 15: end if
- 16: end for

输出: 以 node 为根结点的一棵决策树

决策树算法



决策树学习的关键是算法的第8行：贪心地选择最优划分属性

什么样的划分属性是最优的？

我们希望决策树的分支结点所包含的样本尽可能属于同一类别，即结点的“纯度”越来越高，可以高效地从根结点到达叶结点，得到决策结果。



信息熵与信息增益

“信息熵” (information entropy) 是度量样本集合纯度最常用的一种指标. 假定当前样本集合 D 中第 k 类样本所占的比例为 p_k ($k = 1, 2, \dots, |\mathcal{Y}|$), 则 D 的信息熵定义为

$$\text{Ent}(D) = - \sum_{k=1}^{|\mathcal{Y}|} p_k \log_2 p_k . \quad (4.1)$$

$\text{Ent}(D)$ 的值越小, 则 D 的纯度越高.

假定离散属性 a 有 V 个可能的取值 $\{a^1, a^2, \dots, a^V\}$, 若使用 a 来对样本集 D 进行划分, 则会产生 V 个分支结点, 其中第 v 个分支结点包含了 D 中所有在属性 a 上取值为 a^v 的样本, 记为 D^v . 我们可根据式(4.1) 计算出 D^v 的信息熵, 再考虑到不同的分支结点所包含的样本数不同, 给分支结点赋予权重 $|D^v|/|D|$, 即样本数越多的分支结点的影响越大, 于是可计算出用属性 a 对样本集 D 进行划分所获得的“信息增益” (information gain)

$$\text{Gain}(D, a) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v) . \quad (4.2)$$

信息熵示例



举例：求解划分根结点的最优划分属性

数据集包含17个训练样例：

8个正例（好瓜）占 $p_1 = \frac{8}{17}$

对于二分类任务

9个反例（坏瓜）占 $p_2 = \frac{9}{17}$

$$|y| = 2$$

以属性“色泽”为例计算其信息增益

信息增益示例



用“色泽”将根结点划分后获得3个分支结点的信息熵分别为：

$$\text{Ent}(D^1) = - \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) = 1.000 ,$$

$$\text{Ent}(D^2) = - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.918 ,$$

$$\text{Ent}(D^3) = - \left(\frac{1}{5} \log_2 \frac{1}{5} + \frac{4}{5} \log_2 \frac{4}{5} \right) = 0.722 ,$$

属性“色泽”的信息增益为：

$$\begin{aligned} \text{Gain}(D, \text{色泽}) &= \text{Ent}(D) - \sum_{v=1}^3 \frac{|D^v|}{|D|} \text{Ent}(D^v) \\ &= 0.998 - \left(\frac{6}{17} \times 1.000 + \frac{6}{17} \times 0.918 + \frac{5}{17} \times 0.722 \right) \\ &= 0.109 . \end{aligned}$$

基于信息增益的决策树示例



用“色泽”将根结点划分后获得3个分支结点的信息熵分别为：

类似的，我们可计算出其他属性的信息增益：

$$\text{Gain}(D, \text{根蒂}) = 0.143; \quad \text{Gain}(D, \text{敲声}) = 0.141;$$

$$\text{Gain}(D, \text{纹理}) = 0.381; \quad \text{Gain}(D, \text{脐部}) = 0.289;$$

$$\text{Gain}(D, \text{触感}) = 0.006.$$

基于信息增益的决策树示例

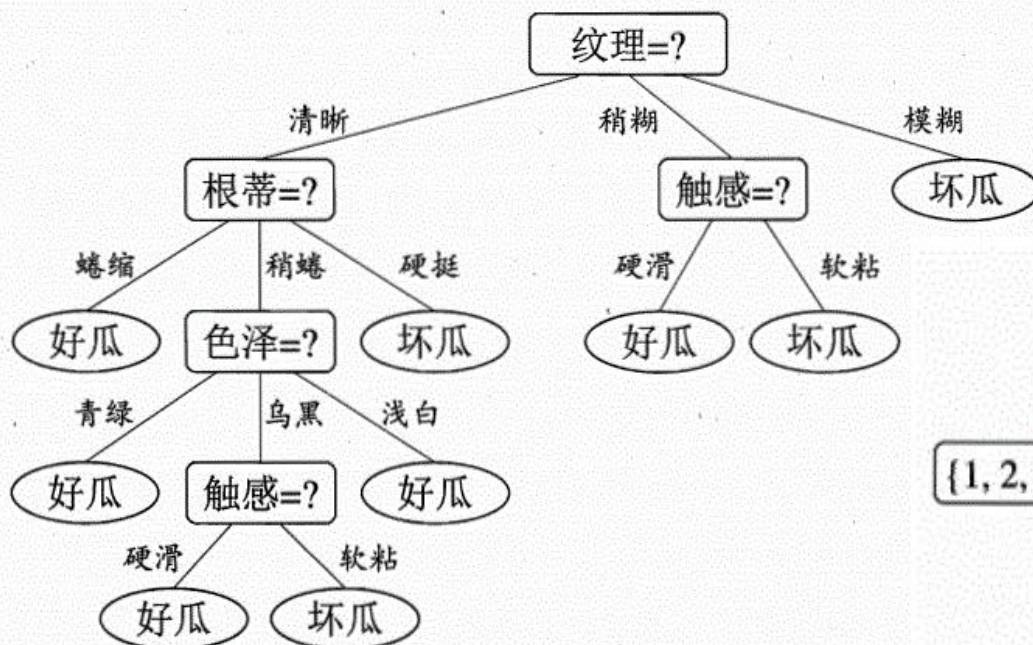


图 4.4 在西瓜数据集 2.0 上基于信息增益生成的决策树



图 4.3 基于“纹理”属性对根结点划分

信息增益的不足



若把“编号”也作为一个候选划分属性，则属性“编号”的信息增益为：

根结点的信息熵仍为： $Ent(D) = 0.998$

用“编号”将根结点划分后获得17个分支结点的信息熵均为：

$$Ent(D^1) = \dots = Ent(D^{17}) = -\left(\frac{1}{1} \log_2 \frac{1}{1} + \frac{0}{1} \log_2 \frac{0}{1}\right) = 0$$

则“编号”的信息增益为：

$$Gain(D, \text{编号}) = Ent(D) - \sum_{v=1}^{17} \frac{1}{17} Ent(D^v) = 0.998$$

远大于其他候选属性

信息增益准则对可取值数目较多的属性有所偏好

增益率



$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(a)}, \quad (4.3)$$

其中

$$\text{IV}(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|} \quad (4.4)$$

称为属性 a 的“固有价值” (intrinsic value) [Quinlan, 1993]. 属性 a 的可能取值数目越多(即 V 越大), 则 $\text{IV}(a)$ 的值通常会越大.

著名的C4.5决策树算法综合了**信息增益准则**和**信息增益率**的特点：先从候选划分属性中找出信息增益高于平均水平的属性，再从中选择增益率最高的。

聚类算法典型——K均值算法



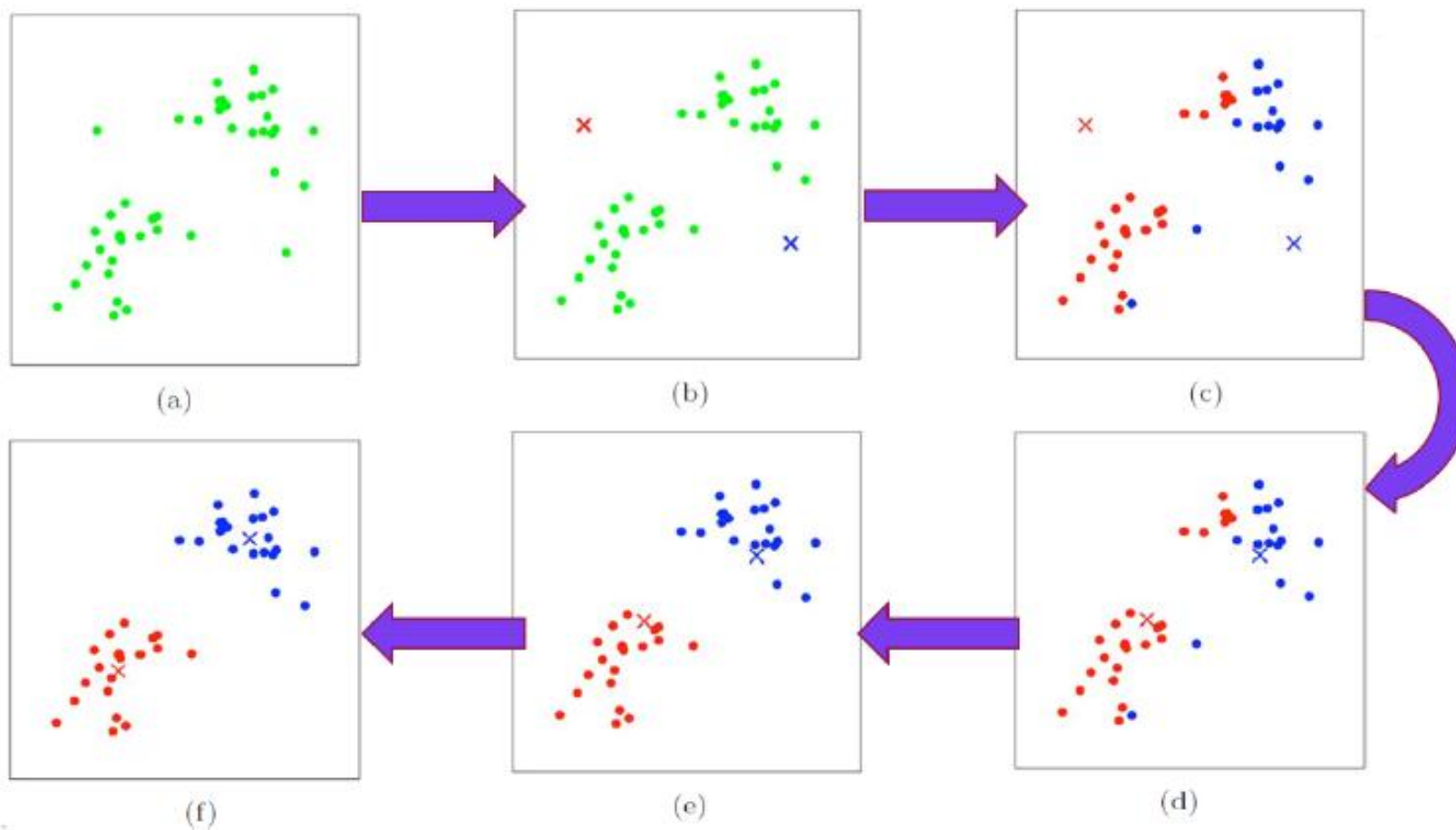
k-均值 (k-means) 算法，是一种得到最广泛使用的聚类算法
它将各个聚类子集内的所有数据样本的均值作为该聚类的代表点，其他点和其最近的代表点同一个类

K均值算法过程



1. 为每个聚类随机确定一个实例作为初始聚类中心，这样就有K个初始聚类中心
2. 将每个实例按照最小距离原则分配到最邻近聚类（即贪心选择）
3. 使用每个聚类中实例均值作为新聚类中心
4. 重复步骤2-3直到聚类中心不再变化
5. 结束，得到K个聚类

算法运行实例



K均值算法距离定义



距离定义：典型定义为欧式距离：

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

K均值算法特点

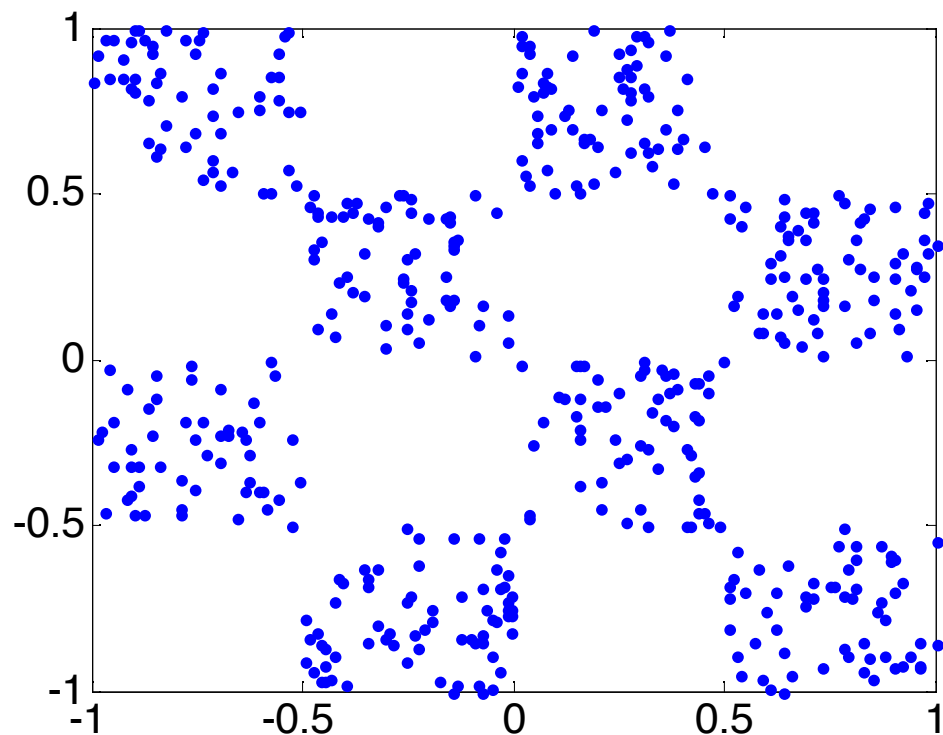


- 必须事先给出 k （要生成的簇的数目）；
- 对初值敏感；
- 对于“噪声”和孤立点数据是敏感的，少量的该类实例能够对结果产生极大的影响；
- 不能保证找到全局最优解，只能确保局部最优解

初始中心的选取对算法的影响



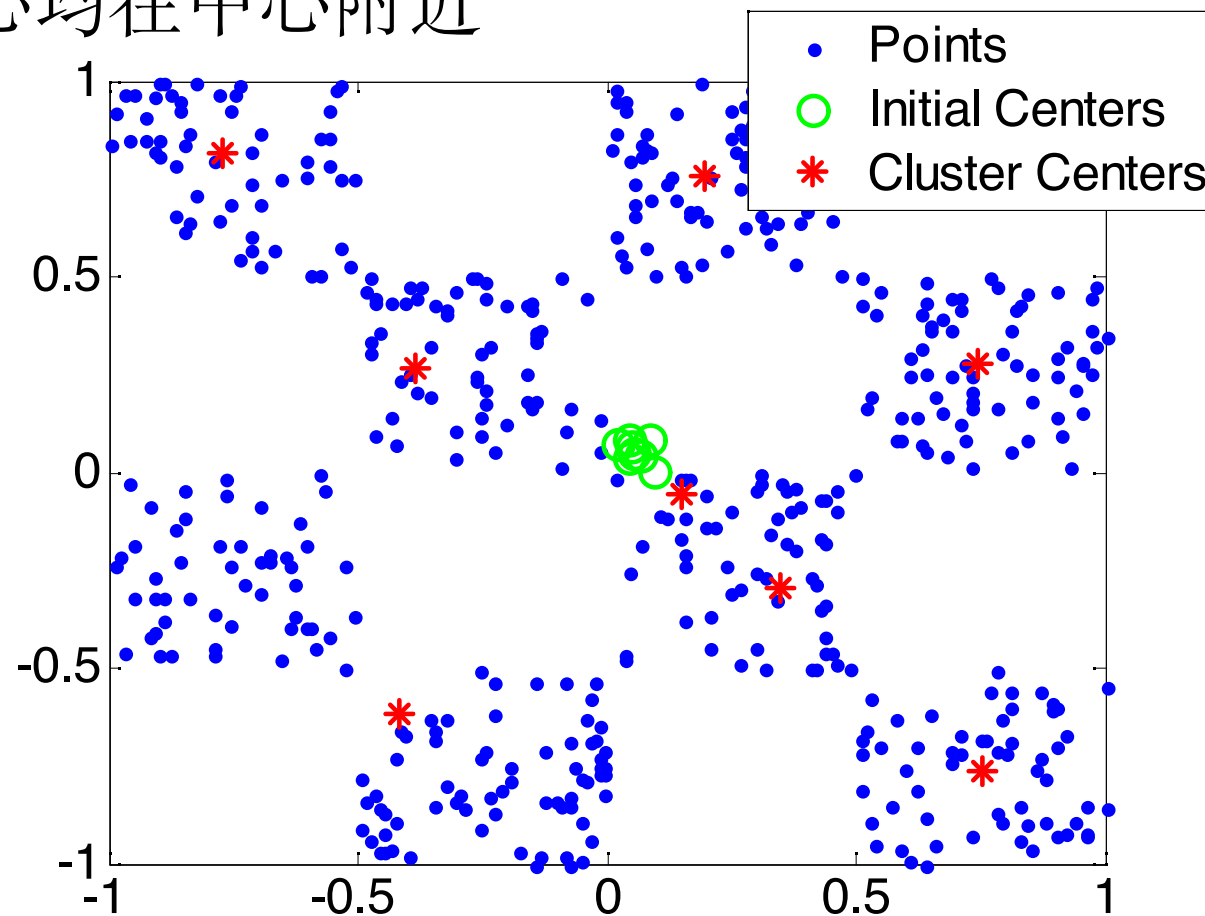
棋盘格数据集(Checkerboard data set)



初始中心的选取对算法的影响



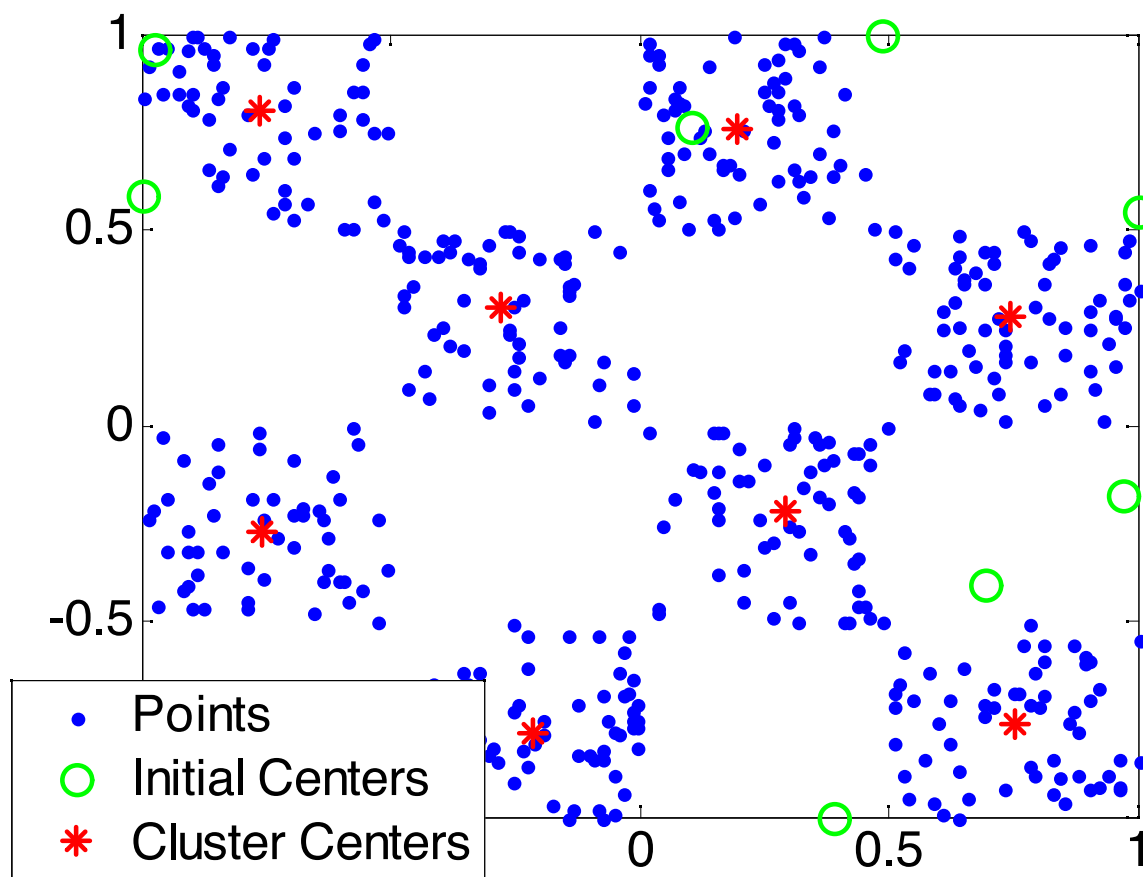
初始聚类中心均在中心附近



初始中心的选取对算法的影响



初始聚类中心在平面内随机选取



机器学习参考书目



《机器学习》，周志华著，清华大学出版社

《统计学习方法》，李航著，清华大学出版社



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——